

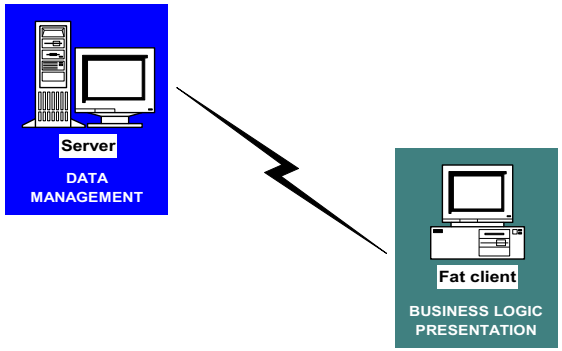
**ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ**
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Τεχνολογίες Υπηρεσιών Λογισμικού

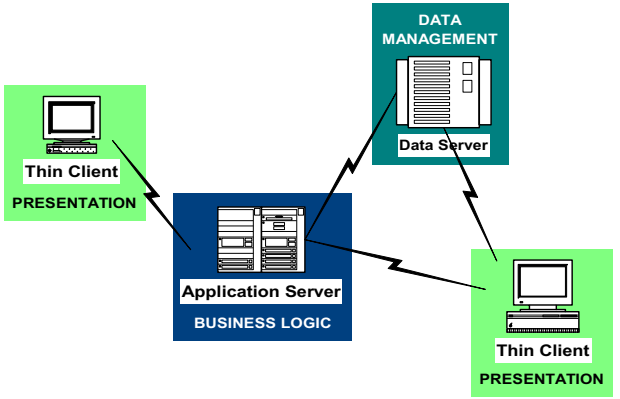
# Τεχνολογίες Υπηρεσιών Λογισμικού

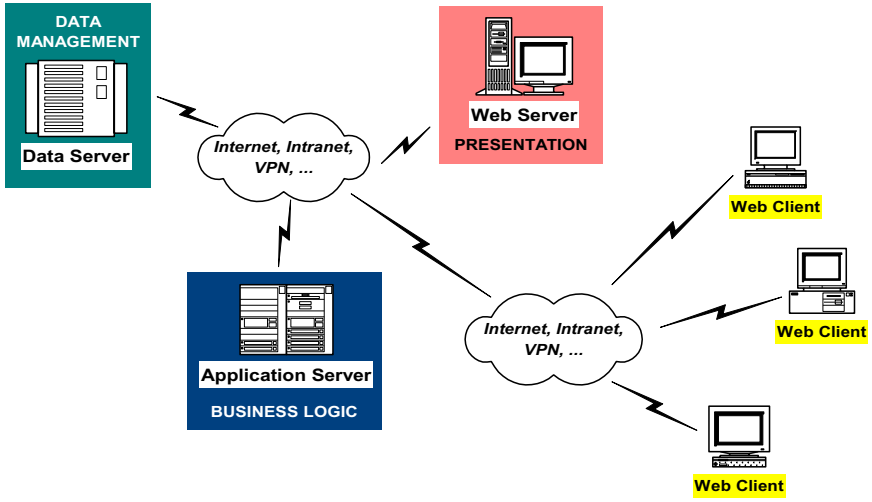
Β.Βεσκούκης, Καθ.ΕΜΠ (v.vescoukis@cs.ntua.gr)

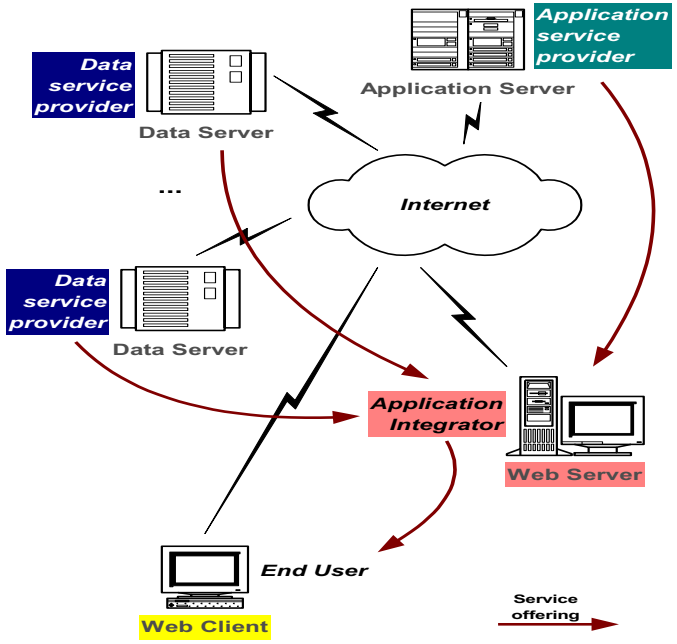
# Web technologies for SaaS: HTTP, HTML, CSS, JSON

# Client-server variants



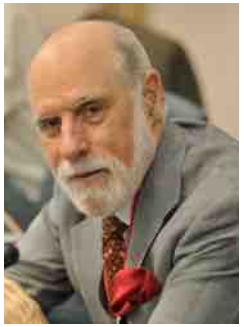



# Web applications/services

## Points to notice

- The web was not designed to host applications
- Security, privacy, transaction integrity, availability, session, connectivity management etc, were no design concerns
- “De-facto” vs “official” standards


Vinton Cerf


Tim Berners-Lee


Robert Kahn

# SaaS is client-server

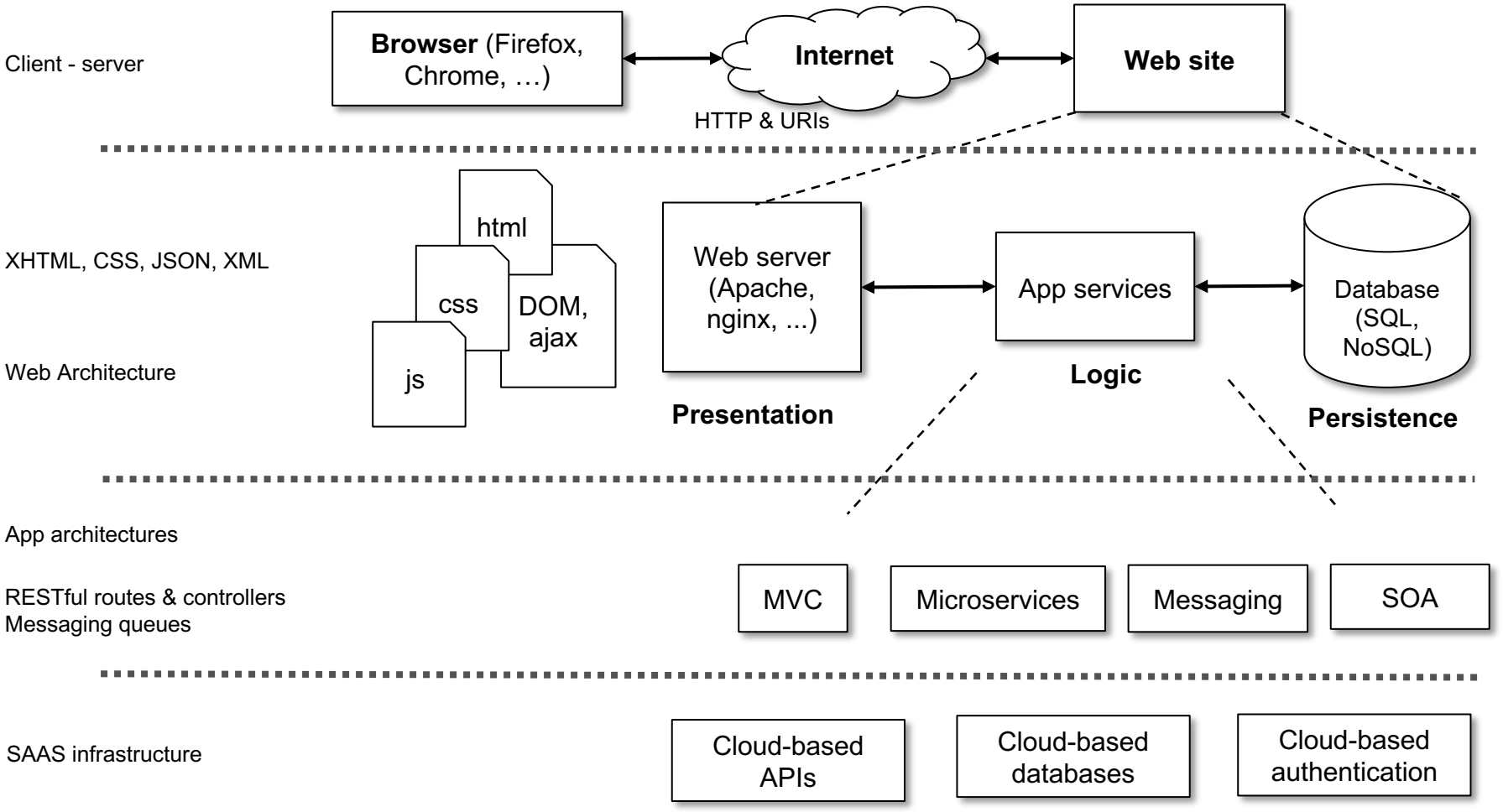
## Load sent from client(s) to server

- Early days: resource id
- Today: (+) ask services to be executed (RPC-type, server-side scripting), pass parameters

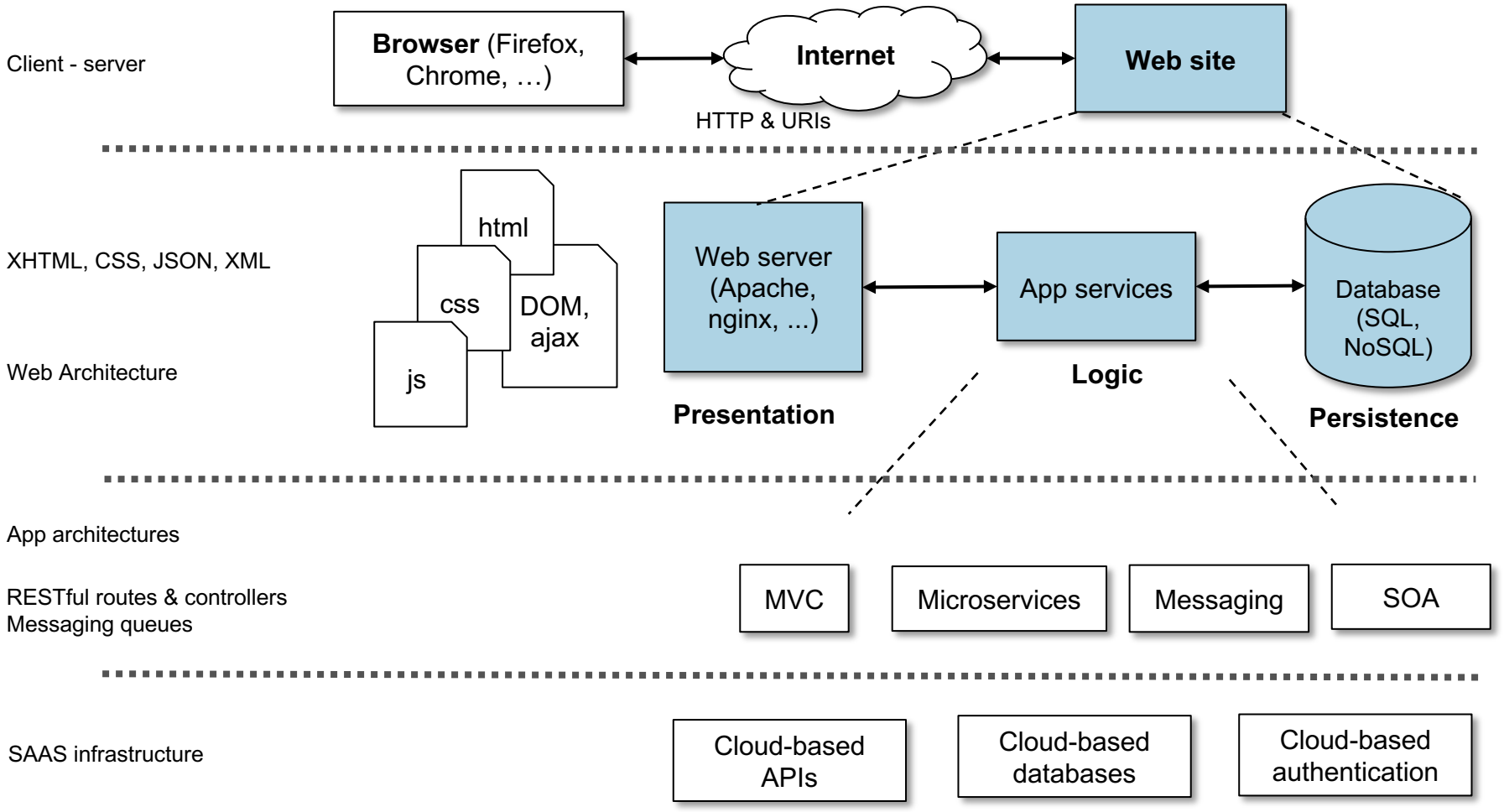
## Load sent from server to client(s)

- Early days: text, formatting, images
- Today: (+) scripts to be executed on the browser (client-side scripting), status info, machine-processable data (e.g. json)

# SaaS course reference technology stack



# Server-side scripting

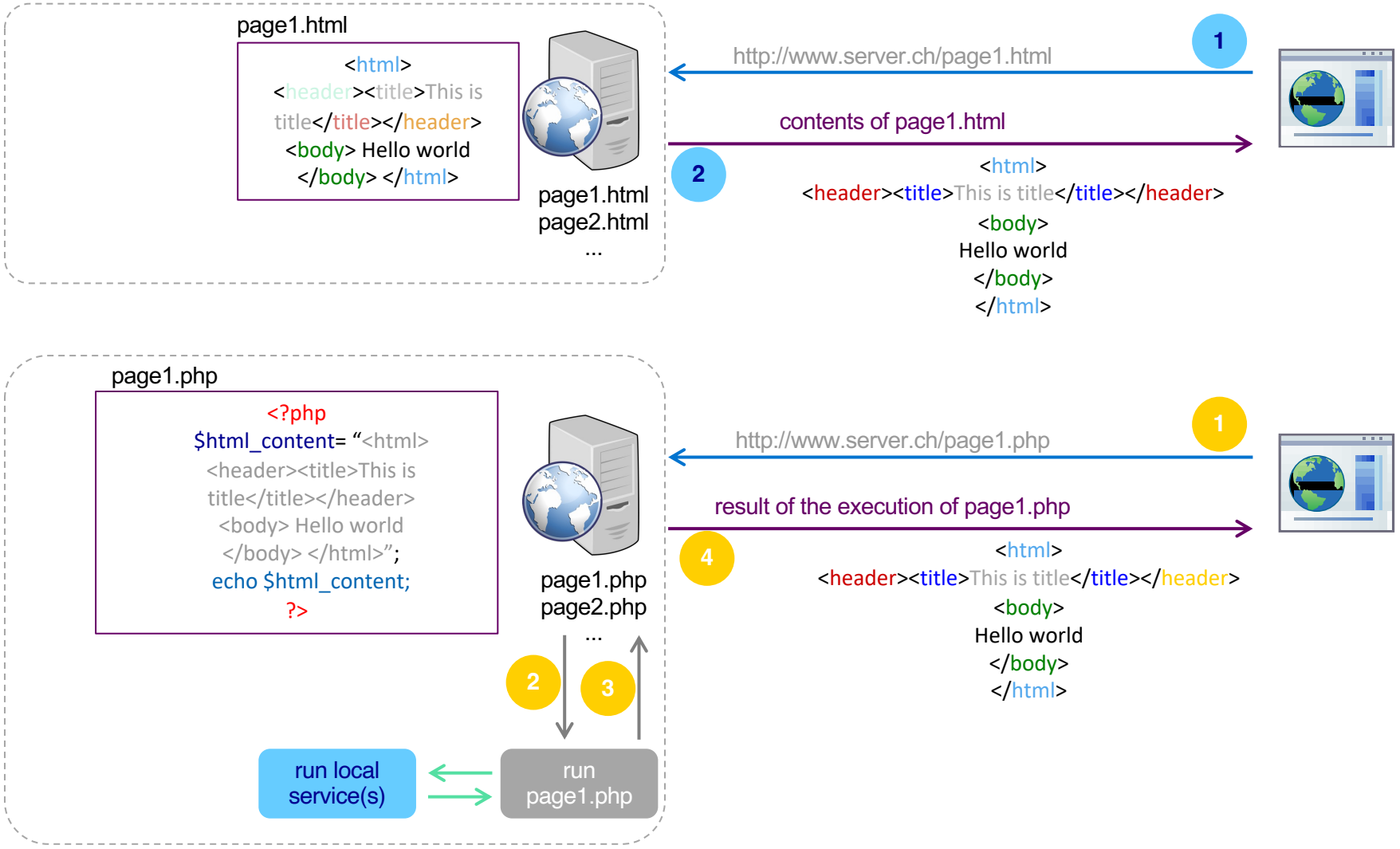


# Server-side scripting

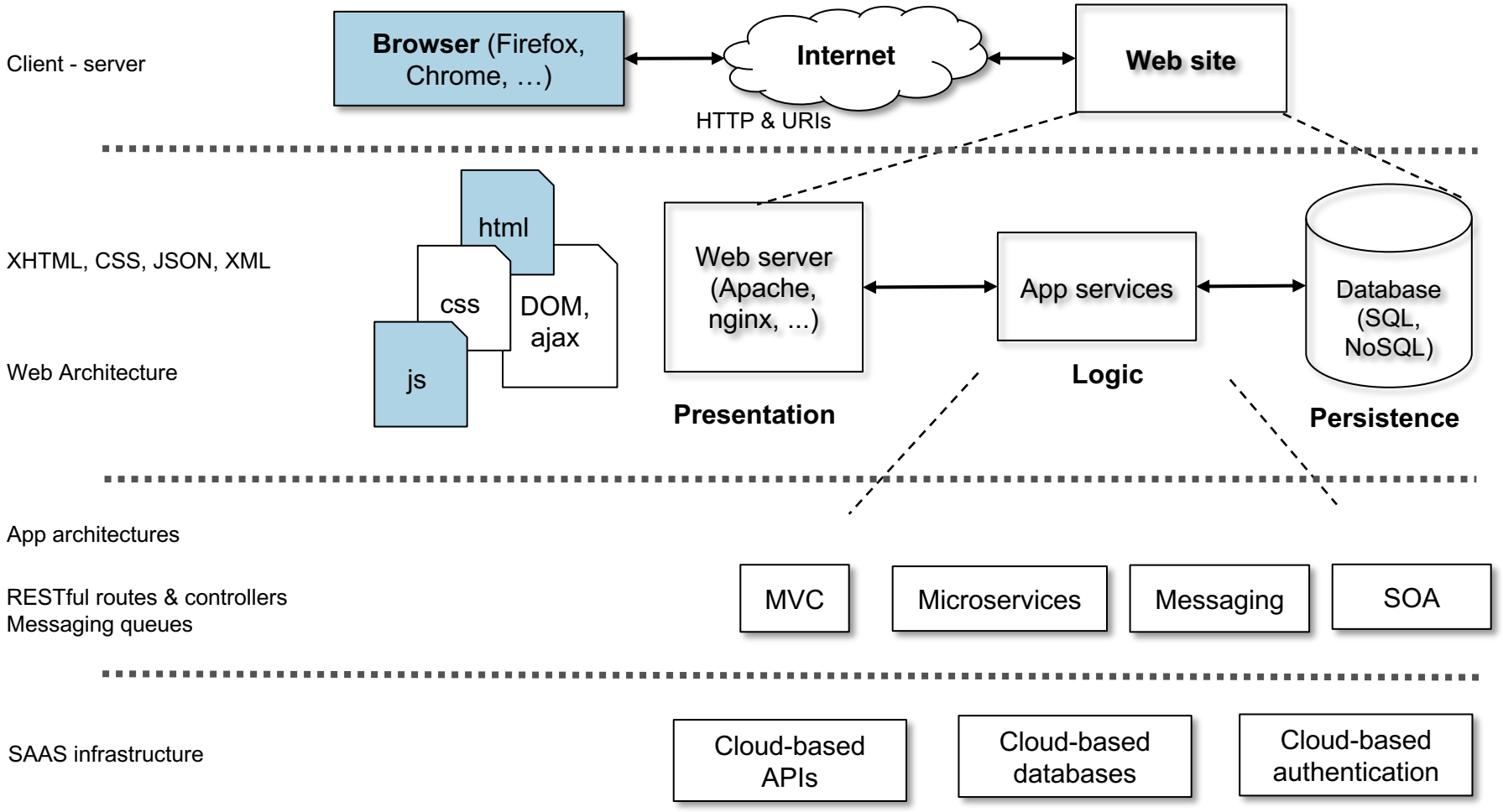
## How server-side scripting works

- Request received by the client
- Server runs locally a program (script)
- The output of the program is HTML or any other text
- The resulting text is sent to the client using HTTP
- The client does not know if the received data comes from a script or is static text
- Scripts have local privileges on the server as needed for connecting to databases or other services

# Server-side scripting



# Client-side scripting



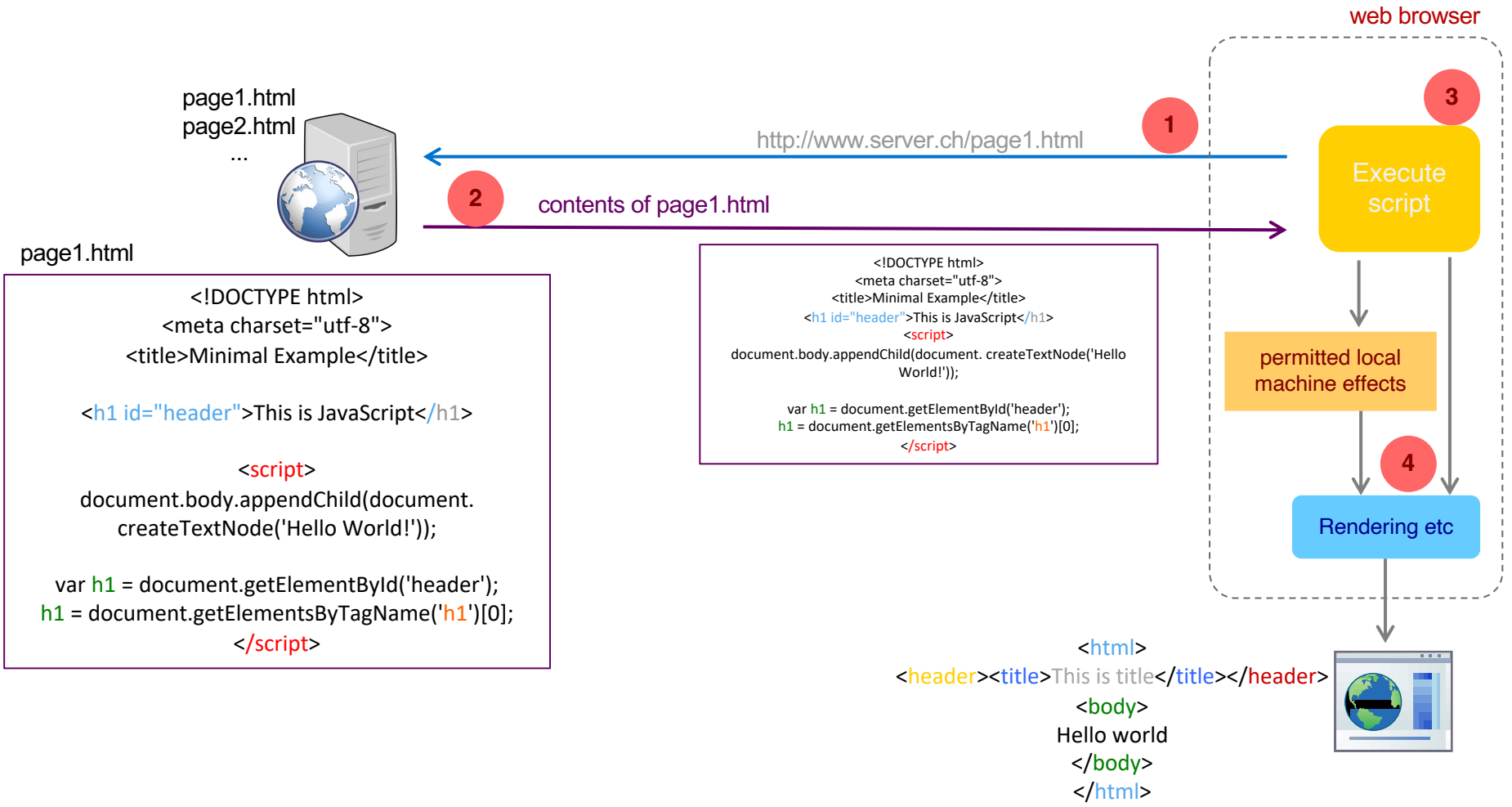
# Client-side scripting

## How client-side scripting works

- Response received by the server contains a recognizable script
- The browser decides where to execute it
- The output of the script is rendered as any other content (HTML is still the host)

- Security warning:
  scripts actually run on the client machine (yours!)

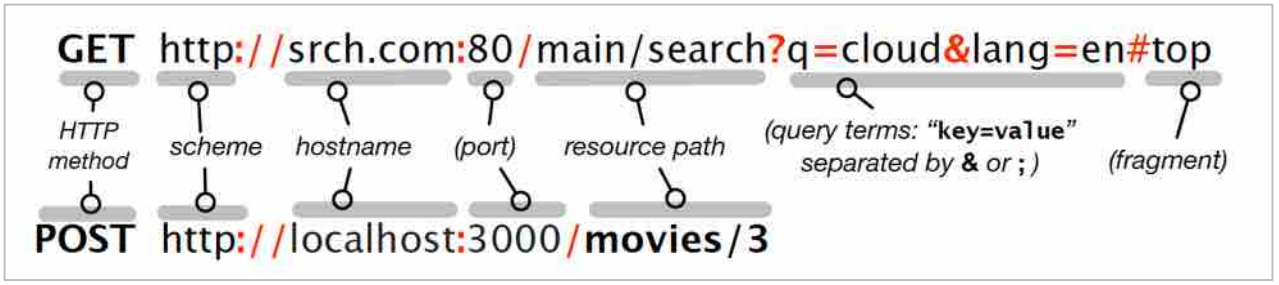
# Client-side scripting (simplified)



# HTTP: the common denominator of SaaS

## HTTP is simple

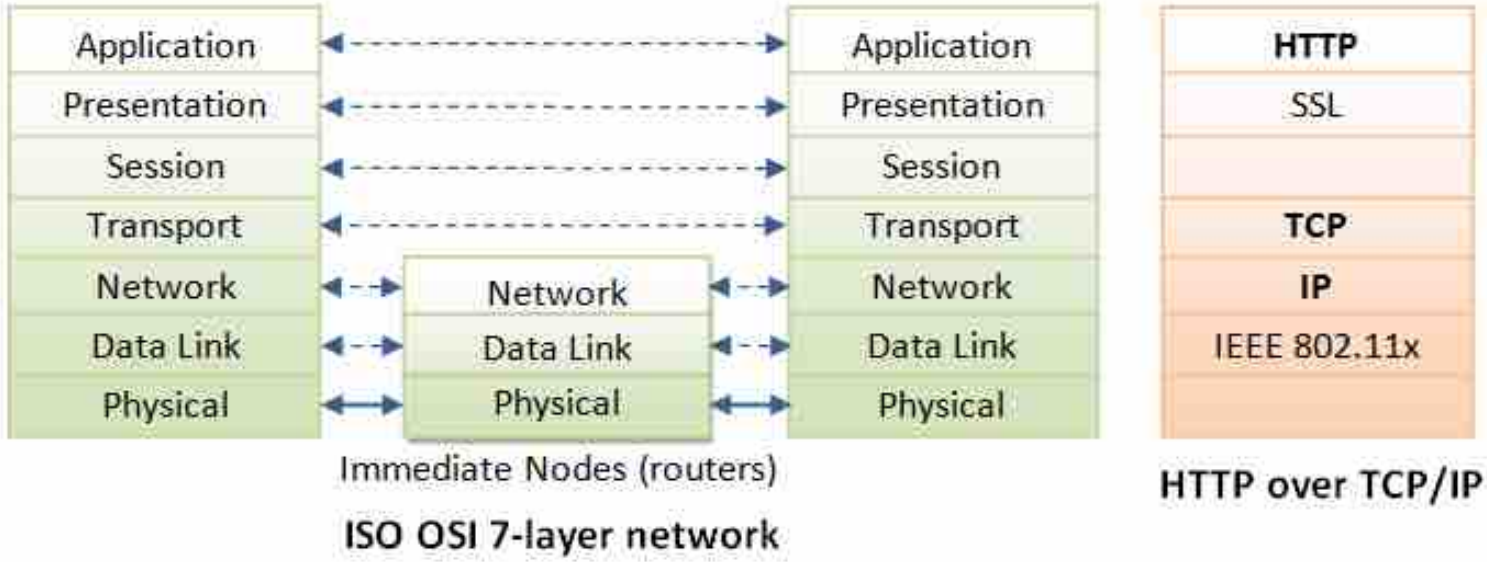
- A client sends a request for a resource, using its URI (rfc 3986, can address within a resource) or URL (rfc 2396, only "location")




- HTTP is stateless (=no link between requests)
- Sessions are possible using cookies
- SaaS *needs* sessions

# HTTP

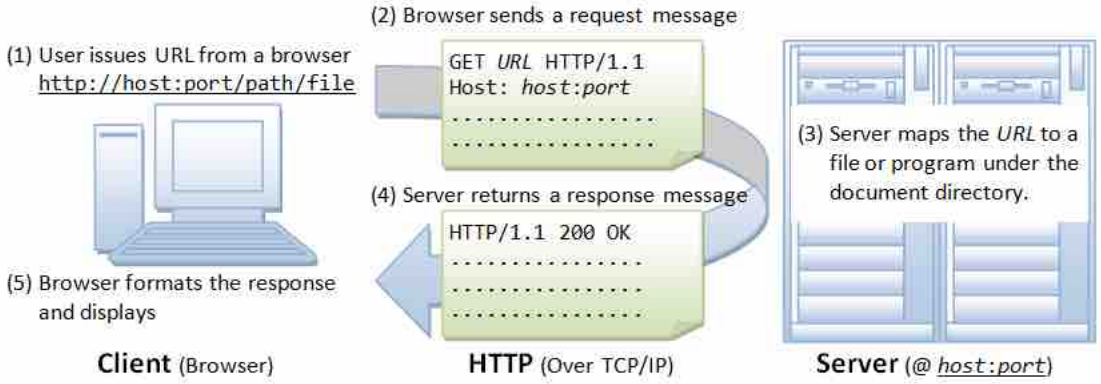
## HTTP and network protocol stack



# HTTP

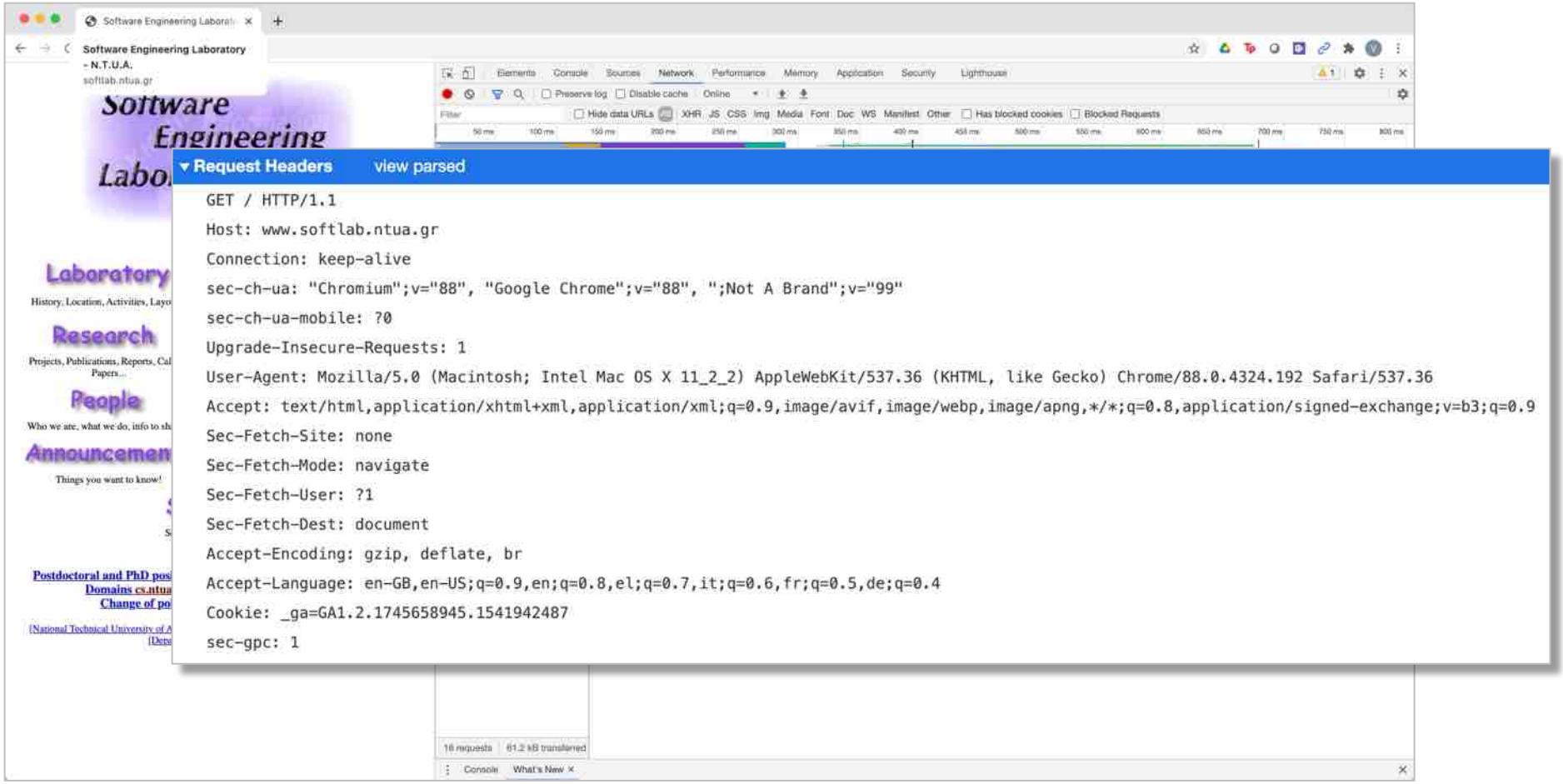
## HTTP requests – responses

- Clients send requests to servers (pull)
- Server send back responses
- A mechanism of request methods, error codes and headers allows HTTP to be used for more than "hypertext"



# HTTP requests

- A URI is translated into a request and sent to the server over the network



# HTTP requests

## Parameter passing is possible via:

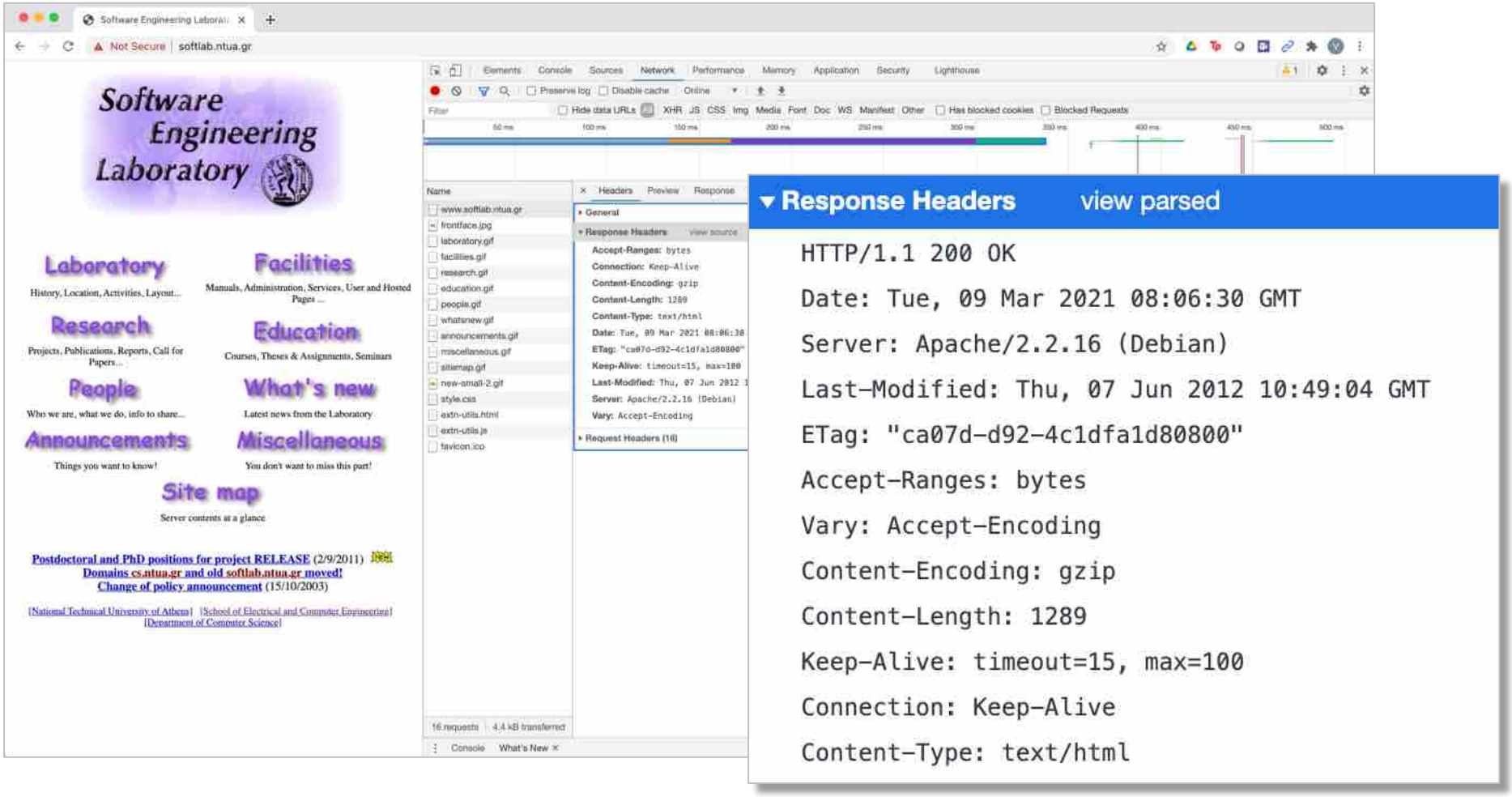
- **query**: softlab.ntua.gr?lang=eng&theme=dark
  - ? — query params follow
  - lang — param1 name
  - eng — param1 value
  - & — param separator
  - theme — param2 name
  - dark — param2 value
- **path**: softlab.ntua.gr/:lang/:theme (definition)
  - :lang — param1 name
  - :theme — param2 name

  softlab.ntua.gr/en/dark (call)
  - en — param1 value
  - dark — param2 value
- **body**: softlab.ntua.gr

  (in body:) { "lang": "en", "theme": "dark" }

# HTTP responses

- The server sends back a response to the request



# HTTP request methods

| | |
|---|---|
| GET | The GET method is used to retrieve information from the given server using a given URI. Requests using GET should only retrieve data and should have no other effect on the data. |
| HEAD | Same as GET, but it transfers the status line and the header section only. |
| POST | A POST request is used to send data to the server, for example, customer information, file upload, etc. using HTML forms. |
| PUT | Replaces all the current representations of the target resource with the uploaded content. Multiple calls do not create multiple new resources (*). |
| DELETE | Removes all the current representations of the target resource given by URI. |
| CONNECT | Establishes a tunnel to the server identified by a given URI. |
| OPTIONS | Describe the communication options for the target resource. |
| TRACE | Performs a message loop back test along with the path to the target resource. |

# GET vs POST

| | GET | POST |
|---|---|---|
| BACK button/Reload | Harmless | Data will be re-submitted (alert by the browser) |
| Bookmarked | Can be bookmarked | Cannot be bookmarked |
| Cached | Can be cached | Not cached |
| Encoding type | application/x-www-form-urlencoded | application/x-www-form-urlencoded or multipart/ form-data. Use multipart encoding for binary data |
| History | Parameters remain in browser history | Parameters are not saved in browser history |
| Restrictions on data length | Yes, when sending data, the GET method adds the data to the URL; and the length of a URL is limited (maximum URL length is 2048 characters) | No restrictions |
| Restrictions on data type | Only ASCII characters allowed | No restrictions |
| Security | Less secure than POST because data sent is part of the URL. Not to be used for sending passwords! | POST is a little safer than GET because the parameters are not stored in browser history or in web server logs |
| Visibility | Data is visible to everyone in the URL | Data is not displayed in the URL |

POST can be used anywhere instead of GET; It is disputable, however, if this is a good practice

# HTTP response status codes

## Defined in rfc 2616

- Informational responses (100–199)
- Successful responses (200–299)
- Redirects (300–399)
- Client errors (400–499)
- Server errors (500–599)

Details:

developer.mozilla.org/en-US/docs/Web/HTTP/Status

# HTTP response status codes

**100** Continue: The server received the request and in the process of giving the response.

**200** OK: The request is fulfilled.

**301** Move Permanently: The resource requested for has been permanently moved to a new location. The URL of the new location is given in the response header called Location. The client should issue a new request to the new location. Application should update all references to this new location.

**302** Found & Redirect (or Move Temporarily): Same as 301, but the new location is temporarily in nature. The client should issue a new request, but applications need not update the references.

**304** Not Modified: In response to the If-Modified-Since conditional GET request, the server notifies that the resource requested has not been modified.

**400** Bad Request: Server could not interpret or understand the request, probably syntax error in the request message.

**401** Authentication Required: The requested resource is protected, and require client's credential (username/password). The client should re-submit the request with his credential (username/password).

**403** Forbidden: Server refuses to supply the resource, regardless of identity of client.

**404** Not Found: The requested resource cannot be found in the server.

**405** Method Not Allowed: The request method used, e.g., POST, PUT, DELETE, is a valid method. However, the server does not allow that method for the resource requested.

**408** Request Timeout

**414** Request URI too Large

**500** Internal Server Error: Server is confused, often caused by an error in the server-side program responding to the request.

**501** Method Not Implemented: The request method used is invalid (could be caused by a typing error, e.g., "GET" misspell as "Get").

**502** Bad Gateway: Proxy or Gateway indicates that it receives a bad response from the upstream server.

**503** Service Unavailable: Server cannot response due to overloading or maintenance. The client can try again later.

**504** Gateway Timeout: Proxy or Gateway indicates that it receives a timeout from an upstream server.

# HTTP headers

## General headers

- apply to both requests and responses, but with no relation to the data transmitted in the body.

## Request headers

- contain more information about the resource to be fetched, or about the client requesting the resource.

# HTTP headers

## Response headers

- hold additional information about the response, like its location or about the server providing it.

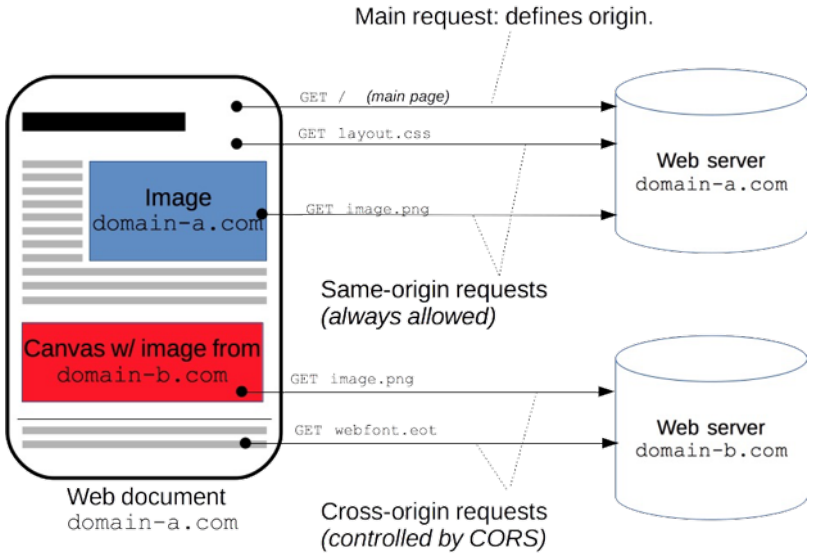
## Entity headers

- contain information about the body of the resource, like its content length or MIME type.

# Cross-Origin Resource Sharing (CORS)

## CORS

- allows a server to indicate any other origins (domain, port) than its own, from which a browser should permit loading of resources
- CORS failures result in errors, *error details are not available to JavaScript*




Further reading

developer.mozilla.org/en-US/docs/Web/HTTP/CORS

# Cross-Origin Resource Sharing (CORS)

## CORS Requests

- Invocations of the XMLHttpRequest or Fetch APIs, as discussed above.
- Web Fonts, so that fonts that can only be cross-site loaded and used when permitted.
- WebGL textures.
- Images/video frames drawn to a canvas using drawImage().
- CSS Shapes from images.

# CORS example

client

```
fetch('https://api.example.com/data')
  .then(r => r.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

server

```
app.get('/data', (req, res) => {
  // ❌ No CORS headers set
  res.json({ message: 'Hello' });
});
```

Console error

```
Access to fetch at
'https://api.example.com/data'
from origin
'https://your-app.com'
has been blocked by CORS policy:
No 'Access-Control-Allow-Origin'
header is present on the
requested resource.
```

# CORS example

## client

```
fetch('https://api.example.com/data')
  .then(r => r.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
```

Response headers + data

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin:
https://your-app.com
Access-Control-Allow-Credentials:
true

{ "message": "Hello" }
```

## server

```
app.get('/data', (req, res) => {
  // ✅ Allow the specific origin
  res.setHeader(
    'Access-Control-Allow-Origin',
    'https://your-app.com'
  );
  // ✅ Allow credentials (cookies)
  res.setHeader(
    'Access-Control-Allow-Credentials',
    'true'
  );
  res.json({ message: 'Hello' });
});
```

# CORS example

server

```
app.get('/data', (req, res) => {
  // ✅ Allow the specific origin
  res.setHeader(
    'Access-Control-Allow-Origin',
    'https://your-app.com'
  );
  // ✅ Allow credentials (cookies)
  res.setHeader(
    'Access-Control-Allow-Credentials',
    'true'
  );
  res.json({ message: 'Hello' });
});
```

```
app.options('/data', (req, res) => {
  res.setHeader(
    'Access-Control-Allow-Origin',
    'https://your-app.com'
  );
  res.setHeader(
    'Access-Control-Allow-Methods',
    'GET, POST, PUT, DELETE'
  );
  res.setHeader(
    'Access-Control-Allow-Headers',
    'Content-Type, Authorization'
  );
  res.sendStatus(204);
});
```

- `Access-Control-Allow-Origin:*` for public read-only APIs
- `withCredentials:true` on the client -> the server must echo back the exact origin (wildcards rejected)

# Further reading

- developer.mozilla.org/en-US/docs/Web/HTTP
- www.postman.com

# HTML

## HTML is the common language for SaaS (latin: "lingua franca")

- Provides space for more than it was designed for...
- Documents are hierarchically structured
- Tags and properties assigned to content
- Classification and id of elements

## HTML history

- 1 (1991), 2 (1995), 3, 4 (1997)
- HTML5 (2014)



# HTML & CSS

## Cascading Style Sheets

- Separate structure from appearance:
  the same content's appearance depends on the CSS rule applied, which may be different for each category of client device / browser used for viewing
- Strong demand for new features:
  responsiveness, accessibility & more
- Strong demand for tools to improve productivity & aesthetics across many devices, screen sizes, etc.

# CSS

## Why "cascading"

- Styles are applied in a specific order (=cascade) to selectors (=identified elements in a page)
- end-user (in browser) →

inside tags →

web page →

web site

→ means: "are applied after"

- All rules are applied
- In conflicts, rule(s) applied last prevail

# HTML & CSS

## Typical CSS rules placement

- **External, in .css file** → Entire site
- Internal in .html (not!) → Same page
- Inline with other tags (not!) → Same tag

```
<h2><font color="darkblue">This is a darkblue H2 tag</font></h2>
```

```
selector { property: value; }
```

```
<style>
    h2 { color: darkblue;}
</style>
<h2>This is a darkblue H2 tag</h2>
```

```
h1, h2, h3 { color: darkblue; font-style: italic; }
```

# CSS for SaaS

## HTML classes and IDs

- **class**: a user-defined attribute
- **ID**: similar to class, unique within a document

```
<style type="text/css">
    .important_info {
        background-color: #ff0000; }

    #email_warning {
        text-blink: true;
        font-size: x-large;
    }
</style>

<p> You need to enroll in courses</p>
<p class="important_info"> Please respect the deadlines</p>
<p id="email_warning"> You must enrol with your university email </p>
```

# CSS for SaaS

## Structuring of HTML containers

- SPAN: inline element that holds text
- DIV: block element, forces new line before

## Good practice

- HTML should only define structure
- All formatting should be done using CSS
- Need for *frameworks* to facilitate this!
- Consider Bootstrap for formatting,
  or other (React etc) for automation









# Resources

- W3C Style Examples

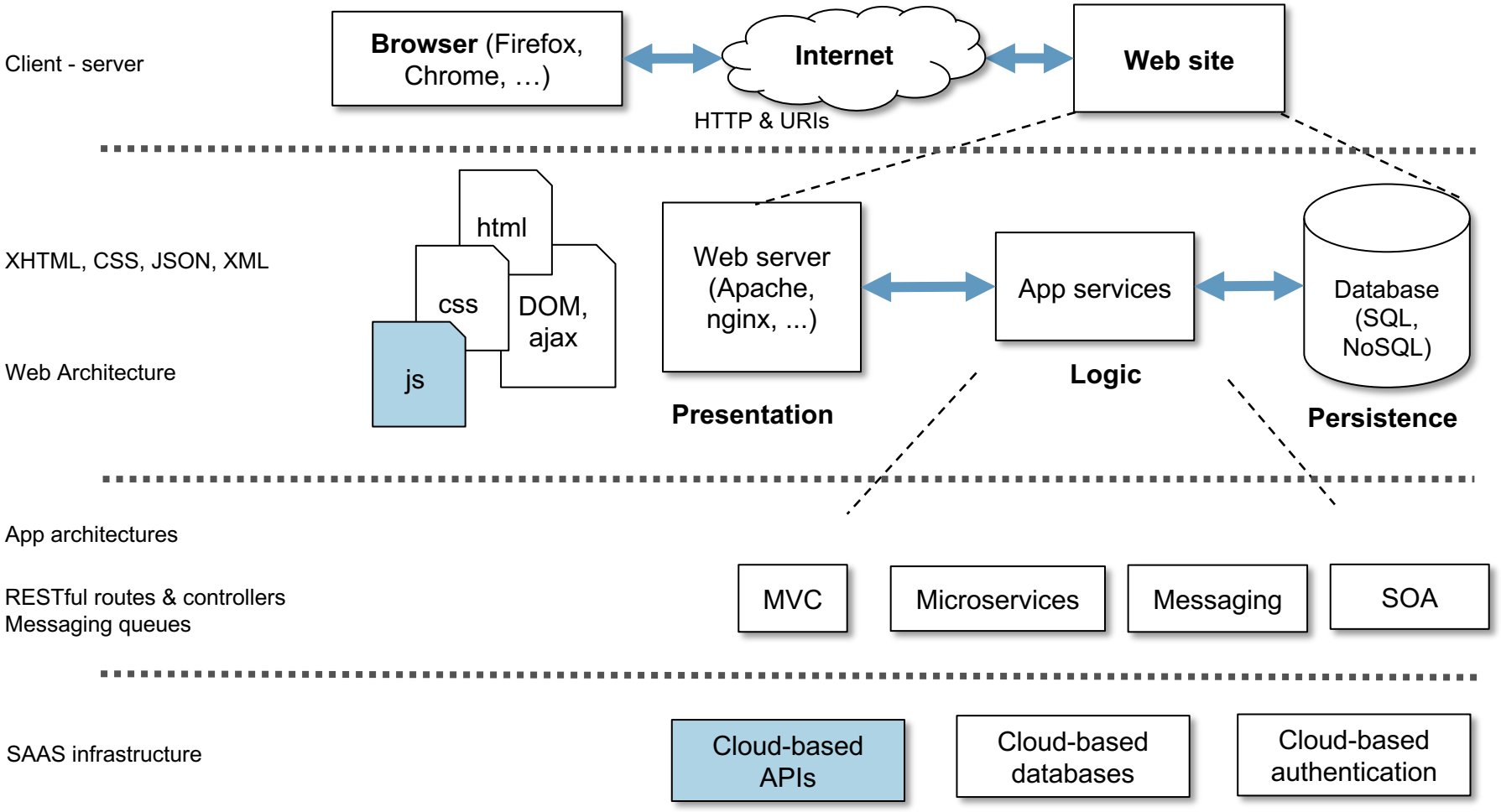
  www.w3.org/Style/Examples/007
- W3C CSS 2.1 Specifications:

  www.w3.org/TR/CSS21/
- W3Schools CSS Tutorial:

  www.w3schools.com/css
- W3Schools CSS Reference:

  www.w3schools.com/css/css_reference.asp

# JSON for data exchange



# A word about JSON

## JSON stands for JavaScript Object Notation

- Very simple to write and parse using Javascript (ECMA-404)
- Efficient and simple structured data communication
- Data types: Numbers, Strings, Booleans, Arrays, Objects, NaN

## JSON-based thematic standards

- **GeoJSON**: Simple JSON format for geographical features: geometries (point, line, polygon etc), features, collections, coordinates
- Google for more...

# A word about JSON

## Minimal and simple

```
{
  "ID" : "123",
  "Last" : "Turing",
  "First" : "Alan",
  "height" : 190
}
```

*Unordered sets of name / value pairs*

```
[ "Apples", "Oranges", "Lemons" ]
```

*Arrays (ordered)*

```
[
  {
    "First" : "Vinton",
    "Last" : "Cerf"
  },
  {
    "First" : "Tim",
    "Last" : "Burners-Lee"
  },
  {
    "First" : "Robert",
    "Last" : "Kahn"
  }
]
```

# A word about JSON

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "Point",
        "coordinates": [102.0, 0.6]
      },
      "properties": {
        "prop0": "value0"
      }
    },
    {
      "type": "Feature",
      "geometry": {
        "type": "LineString",
        "coordinates": [
          [102.0, 0.0], [103.0, 1.0], [104.0, 0.0], [105.0, 1.0]
        ]
      },
      "properties": {
        "prop1": 0.0,
        "prop0": "value0"
      }
    }
  ]
}
```

# GET parameters / JSON parsing

## Parsing URI from GET to create JSON object

- Parameters come in the URI
- Use url.parse of req object (deprecated)

```
http://localhost:8080?month=March&year=2021
```

```
const http = require('http');
const url = require('url');

http.createServer(function (req, res) {
  const queryObject = url.parse(req.url,true).query;
  console.log(queryObject);

  res.writeHead(200, {'Content-Type': 'text/html'});
  res.end('Feel free to add query parameters to the end of the url');
}).listen(8080);
```

https://nodejs.org/en/knowledge/HTTP/clients/how-to-access-query-string-parameters/

# GET parameters / JSON parsing

## Parameter parsing from GET

- Parameters come in the URI
- Parse req.url

```
http://localhost:8080?month=March&year=2021
```

```
const par_string = req.url.substring(2);
let par_object = Object.fromEntries(new URLSearchParams(par_string));
```

# POST parameters / JSON parsing

## POST parameters come into the HTML body

- URI can still contain parameters
- No size limit, no URI encoding issues...

## Strategy:

- Wait for the POST request to be received
- Parse HTML body to create JSON object(s)

```
req.on('data', function (chunk) {
    body += chunk;
});
```



```
req.on('end', function () {
    jsonbody_in = JSON.parse(body);
});
```

# Async frontend: AJAX (+ DOM + jquery)

# The Document Object Model (DOM)

## HTML: a complete description of a web site

- Structure (e.g. header, body, divs)
- Static content, created by an editor
- Dynamic content, created at load/runtime
  either on the server or the client (browser)

## DOM: a representation of an HTML document

- Created by the browser
- Allows the elements of a web page
  to be accessed as objects

# DOM consists of nodes

## Node types

- Element (1), Text (3), Comment(8)
- $0: the active node object ref (c++ "this")

## Access to elements

- .getElementBy { Id, ClassName, Name, TagName }

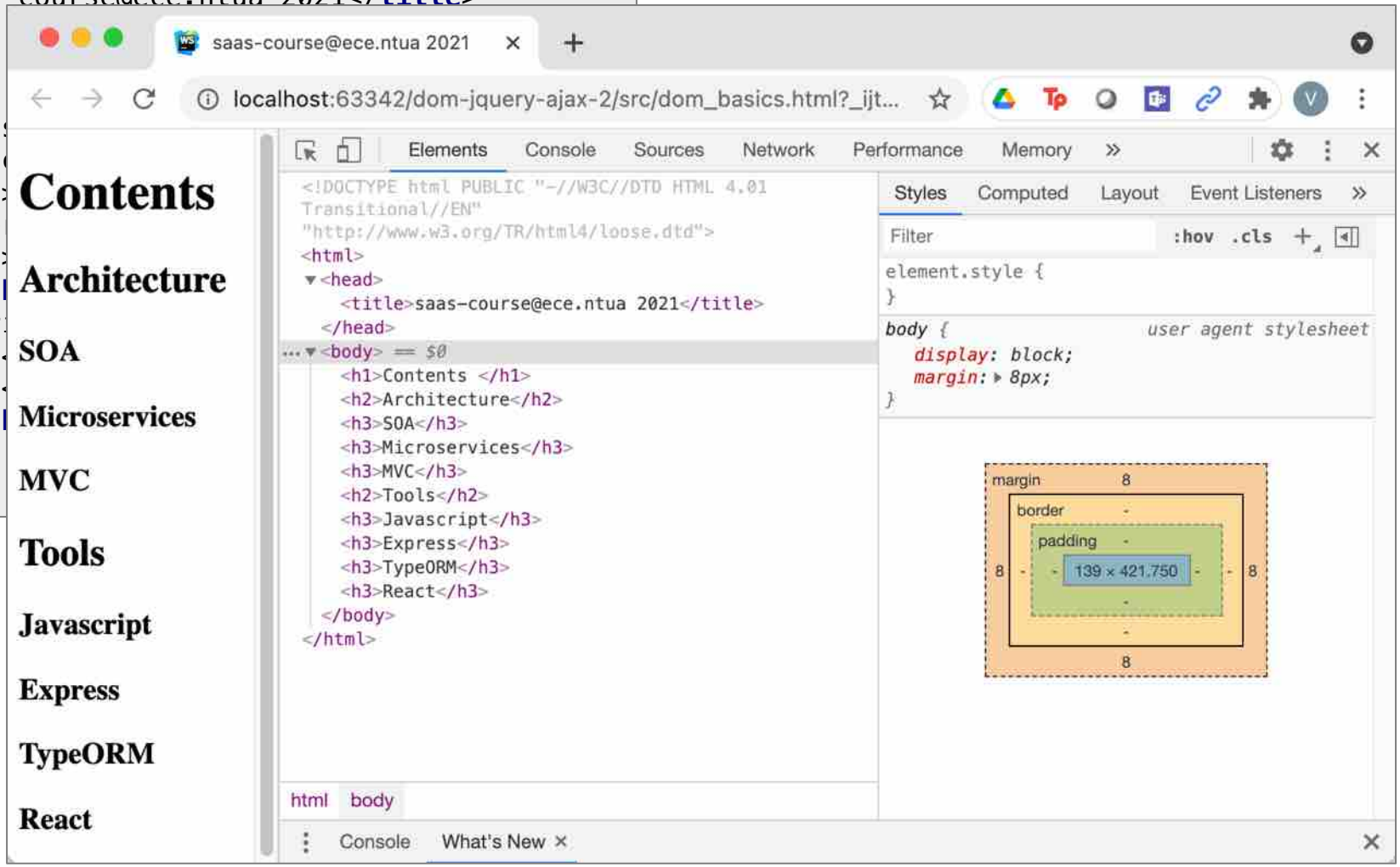
## Use js to modify DOM elements

```
let button1 = document.getElementById('coolButton');
button1.addEventListener('click', () => {
    document.getElementById('intro').style.backgroundColor = 'green';
})
```

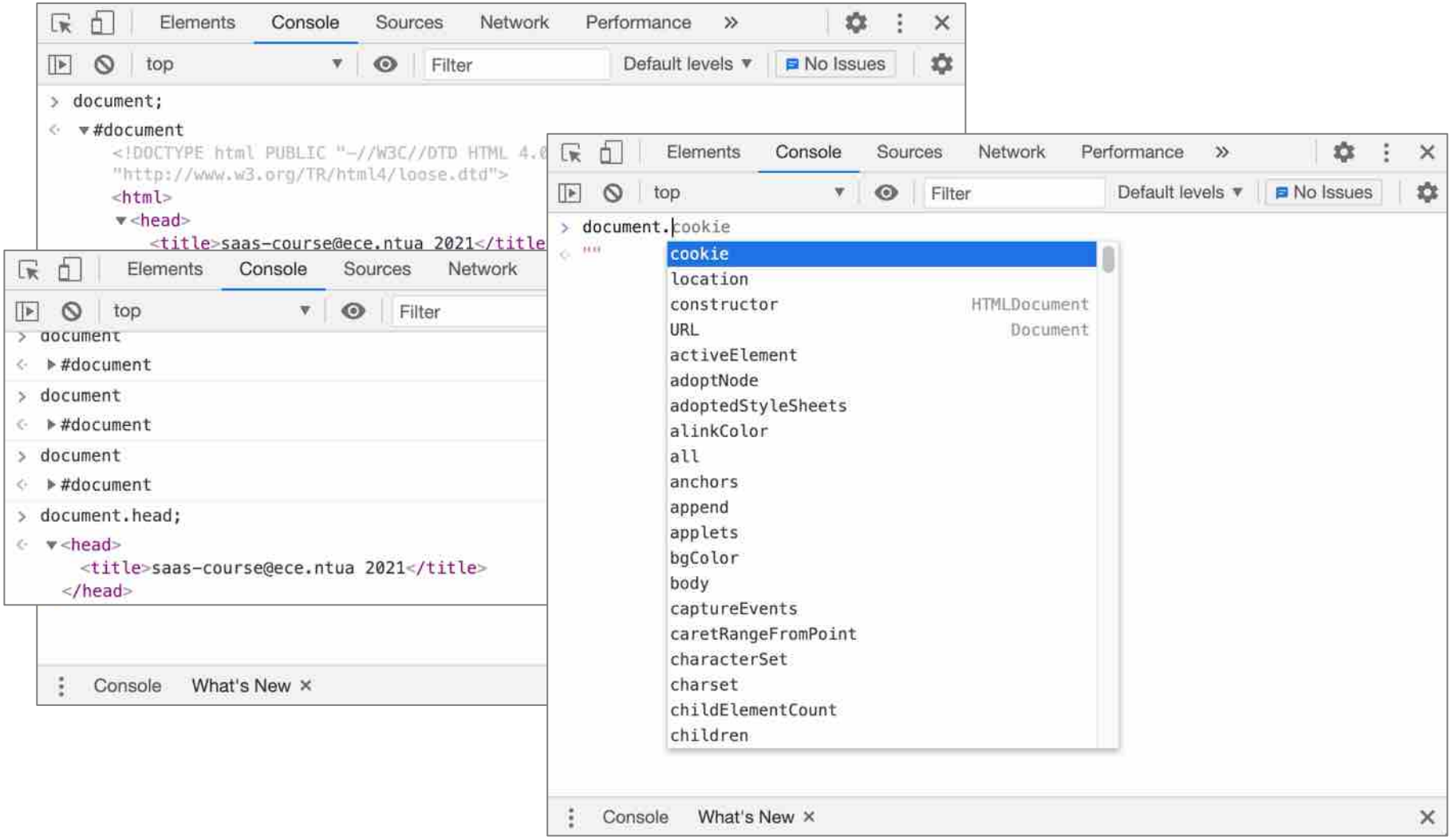
# The Document Object Model

```
<html>
<head>
    <title>saas-course@ece.ntua 2021</title>
</head>

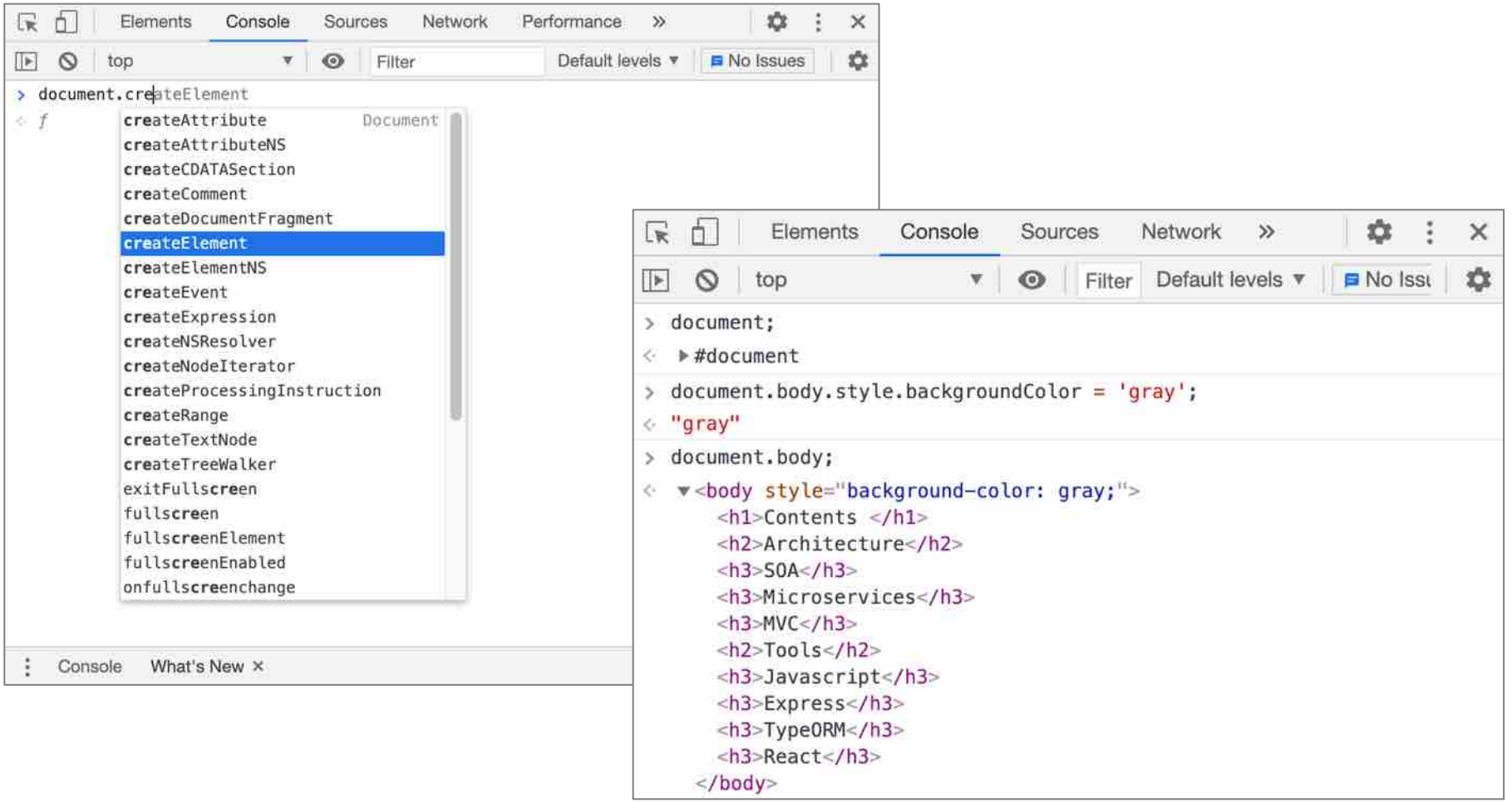
<body>
    <h1>Contents
    <h2>Archite
    <h3>SOA</h3
    <h3>Microse
    <h3>MVC</h3
    <h2>Tools</
    <h3>Javascr
    <h3>Express
    <h3>TypeORM
    <h3>React</
</body>
</html>
```



# The Document Object Model



# The Document Object Model



# The Document Object Model

```
<body>
    <div id="saas content">
        <h1>Contents </h1>
        <h2>Architecture</h2>
        <h3>SOA</h3>
        - <a id='soa1' href="slides.html">Presentation </a>
        <br>
        - <a id='soa2' href="example.html">Example </a>
        <h3>Microservices</h3>
        <h3>MVC</h3>
        <h2>Tools</h2>
        <h3>Javascript</h3>
        <h3>Express</h3>
        <h3>TypeORM</h3>
        <h3>React</h3>
    </div>
</body>
```



# Access to DOM nodes

## Selection & querying

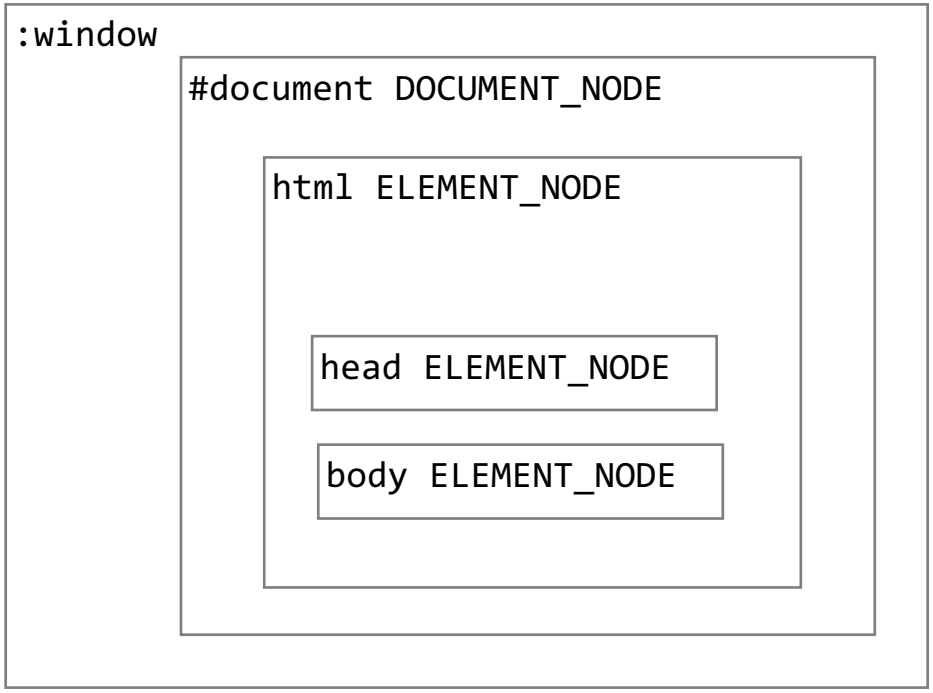
```
let button3 = document.getElementById('bordersButton');

button3.addEventListener('click', () => {
    const level3 = document.querySelectorAll('h3');
    level3.forEach(item => {
        if (item.style.border === '1px solid red')
            item.style.border = 'none';
        else
            item.style.border = '1px solid red';
    });
```

# Traverse the DOM

## Useful properties

- parentNode
- parentElement*
- childNodes
- firstChild
- lastChild
- children*
- firstElementChild*
- lastElementChild*



# Change the DOM

## Create, delete, modify elements

- **create**: createElement(), createTextNode()
- **get/set**: node.textContent, node.innerHTML

## Alter the DOM

- node.{appendChild(), insertBefore(), replaceChild()}
- node.removeChild(), remove()

- You can also modify attributes, classes, styles (hint: rtfm)

```
> const lll = document.querySelector('ul');
<- undefined
> lll;
<- ▶<ul>…</ul>
> const newfish = document.createElement('li');
<- undefined
> newfish.textContent='tsipoura';
<- "tsipoura"
> lll.appendChild(newfish);
<- ▶<li>…</li>
>
```

# Working with the DOM in js: jQuery

## jQuery: what is it

- Javascript library for DOM manipulation

## jQuery: why need it

- Browser incompatibilities in DOM management

**JavaScript**

```
document.getElementById("myHello").innerHTML = "Hello SaaS World!";
```

**jQuery**

```
$("#myHello").html("Hello SaaS World!");
```

# jQuery features

## What jQuery does

- HTML manipulation
- DOM manipulation
- DOM element selection
- CSS manipulation
- Effects and Animations
- AJAX
- HTML event methods
- JSON Parsing

# jQuery

## Setup

- Load jQuery API *before* any js code

```
<html>
  <head>
    <title>SaaS@ntua – jquery</title>
    <script src="https://ajax.googleapis.com/ajax/libs/jquery/3.6.0/jquery.min.js"></script>
    <script src="jqueryscripts.js"></script>
  </head>
  <body>



  </body>
</html>
```

- Use jQuery

```
$(document).ready(function() {
        $("#myContent").html("Hello SaaS world!");
});
```

# jQuery

## jQuery calls: `$("selector").method();`

- Selectors

`$("*")` **Wildcard:** selects every element on the page.

`$(this)` **Current:** selects the current element being operated on within a function.

`$("p")` **Tag:** selects every instance of the <p> tag.

`$(".example")` **Class:** selects *every* element that has the example class applied.

`$("#example")` **Id:** selects a *single instance* of the unique example id.

`$("[type='text']")` **Attribute:** selects any element with *text* applied to the type attribute.

`$("p:first-of-type")` **Pseudo Element:** selects the first <p>.

# jQuery

| Selector | Example | Description |
|---|---|---|
| * | $("*") | It is used to select all elements. |
| #id | $("#firstname") | It will select the element with id="firstname" |
| .class | $(".primary") | It will select all elements with class="primary" |
| class,.class | $(".primary,.secondary") | It will select all elements with the class "primary" or "secondary" |
| element | $("p") | It will select all p elements. |
| el1,el2,el3 | $("h1,div,p") | It will select all h1, div, and p elements. |
| :first | $("p:first") | This will select the first p element |
| :last | $("p:last") | This will select he last p element |
| :even | $("tr:even") | This will select all even tr elements |
| :odd | $("tr:odd") | This will select all odd tr elements |
| :first-child | $("p:first-child") | It will select all p elements that are the first child of their parent |
| :first-of-type | $("p:first-of-type") | It will select all p elements that are the first p element of their parent |
| :last-child | $("p:last-child") | It will select all p elements that are the last child of their parent |
| :last-of-type | $("p:last-of-type") | It will select all p elements that are the last p element of their parent |

# jQuery

| | | |
|---|---|---|
| :nth-child(n) | $("p:nth-child(2)") | This will select all p elements that are the 2nd child of their parent |
| :nth-last-child(n) | $("p:nth-last-child(2)") | This will select all p elements that are the 2nd child of their parent, counting from the last child |
| :nth-of-type(n) | $("p:nth-of-type(2)") | It will select all p elements that are the 2nd p element of their parent |
| :nth-last-of-type(n) | $("p:nth-last-of-type(2)") | This will select all p elements that are the 2nd p element of their parent, counting from the last child |
| :only-child | $("p:only-child") | It will select all p elements that are the only child of their parent |
| :only-of-type | $("p:only-of-type") | It will select all p elements that are the only child, of its type, of their parent |
| parent > child | $("div > p") | It will select all p elements that are a direct child of a div element |
| parent descendant | $("div p") | It will select all p elements that are descendants of a div element |
| element + next | $("div + p") | It selects the p element that are next to each div elements |
| element ~ siblings | $("div ~ p") | It selects all p elements that are siblings of a div element |
| :eq(index) | $("ul li:eq(3)") | It will select the fourth element in a list (index starts at 0) |
| :gt(no) | $("ul li:gt(3)") | Select the list elements with an index greater than 3 |
| :lt(no) | $("ul li:lt(3)") | Select the list elements with an index less than 3 |
| :not(selector) | $("input:not(:empty)") | Select all input elements that are not empty |

# jQuery

| | | |
|---|---|---|
| :header | $(":header") | Select all header elements h1, h2 ... |
| :animated | $(":animated") | Select all animated elements |
| :focus | $(":focus") | Select the element that currently has focus |
| :contains(text) | $(":contains('Hello')") | Select all elements which contains the text "Hello" |
| :has(selector) | $("div:has(p)") | Select all div elements that have a p element |
| :empty | $(":empty") | Select all elements that are empty |
| :parent | $(":parent") | Select all elements that are a parent of another element |
| :hidden | $("p:hidden") | Select all hidden p elements |
| :visible | $("table:visible") | Select all visible tables |
| :root | $(":root") | It will select the document's root element |
| :lang(language) | $("p:lang(de)") | Select all p elements with a lang attribute value starting with "de" |
| [attribute] | $("[href]") | Select all elements with a href attribute |
| [attribute=value] | $("[href='default.htm']") | Select all elements with a href attribute value equal to "default.htm" |
| [attribute!=value] | $("[href!='default.htm']") | It will select all elements with a href attribute value not equal to "default.htm" |
| [attribute$=value] | $("[href$='.jpg']") | It will select all elements with a href attribute value ending with ".jpg" |

# jQuery

| | | |
|---|---|---|
| [attribute\|=value] | $("[title\|='Tomorrow']") | Select all elements with a title attribute value equal to 'Tomorrow', or starting with 'Tomorrow' followed by a hyphen |
| [attribute^=value] | $("[title^='Tom']") | Select all elements with a title attribute value starting with "Tom" |
| [attribute~=value] | $("[title~='hello']") | Select all elements with a title attribute value containing the specific word "hello" |
| [attribute*=value] | $("[title*='hello']") | Select all elements with a title attribute value containing the word "hello" |

# jQuery

| | | |
|---|---|---|
| :input | $(":input") | It will select all input elements |
| :text | $(":text") | It will select all input elements with type="text" |
| :password | $(":password") | It will select all input elements with type="password" |
| :radio | $(":radio") | It will select all input elements with type="radio" |
| :checkbox | $(":checkbox") | Itwill select all input elements with type="checkbox" |
| :submit | $(":submit") | It will select all input elements with type="submit" |
| :reset | $(":reset") | It will select all input elements with type="reset" |
| :button | $(":button") | It will select all input elements with type="button" |
| :image | $(":image") | It will select all input elements with type="image" |
| :file | $(":file") | It will select all input elements with type="file" |
| :enabled | $(":enabled") | Select all enabled input elements |
| :disabled | $(":disabled") | It will select all disabled input elements |
| :selected | $(":selected") | It will select all selected input elements |
| :checked | $(":checked") | It will select all checked input elements |

# jQuery get/set properties

## Return property values

```
$("selector").html();
$("selector").text();
$("selector").val();
$("selector").css("propertyName");
```

## Set property values

```
$("selector").html(content);
$("selector").text(content);
$("selector").val(newVal);
$("selector").css("propertyName", "newVal");
```

# jQuery events

- Events

| | |
|---|---|
| `click()` | Executes on a single mouse click. |
| `hover()` | Executes when the mouse is hovered over an element. |
| `submit()` | Executes when a form is submitted. |
| `scroll()` | Executes when the screen is scrolled. |
| `keydown()` | Executes when you press down on a key on the keyboard. |

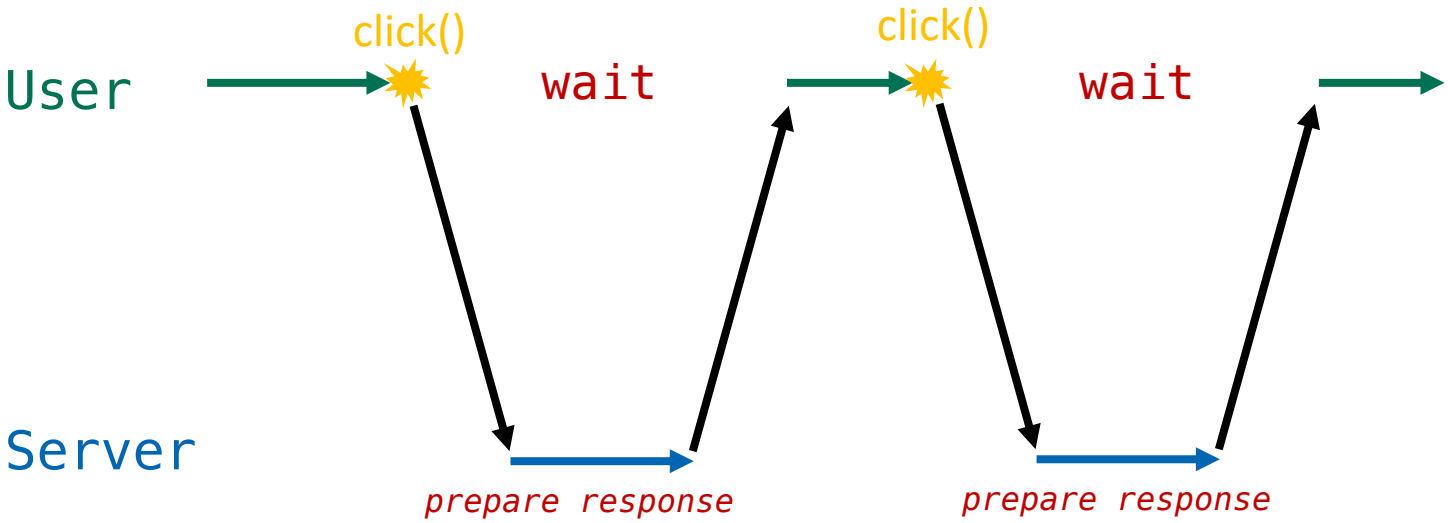
- More events

`bind()`, `unbind()` , `blur()`, `focus()`, `select()`, `change()`, `keypress()`, `keyup()`, `mouseenter()`, `mouseleave()`, `mousedown()`, `mouseup()`, `mouseover()`, `mouseout()`, `load()`, `unload()`, `delegate()`

# Ajax: **A**synchronous **J**avasctipt **A**nd **X**ML

## Synchronous web communication

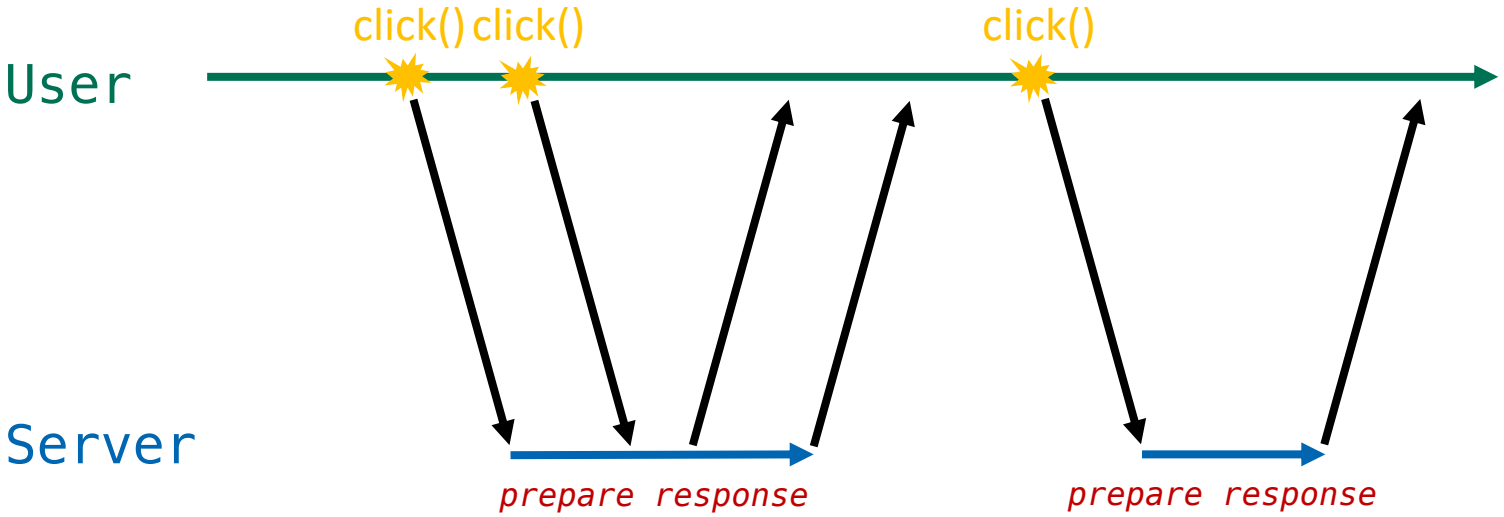
- Click ->
- Wait ->
- Refresh page



# Ajax: **A**synchronous **J**avasctipt **A**nd **X**ML

## Asynchronous web communication

- Click ->
- Continue working ->
- Refresh affected page element



# Ajax: **A**synchronous **J**avasctipt **A**nd **X**ML

## Ajax

- a way of using JavaScript
- downloads data from a server in the background
- allows dynamically updating (elements of) a page without making the user wait
- avoids the "click-wait-refresh" pattern

## Important

- Dependencies between elements / requests should be handled very carefully!

# XMLHttpRequest

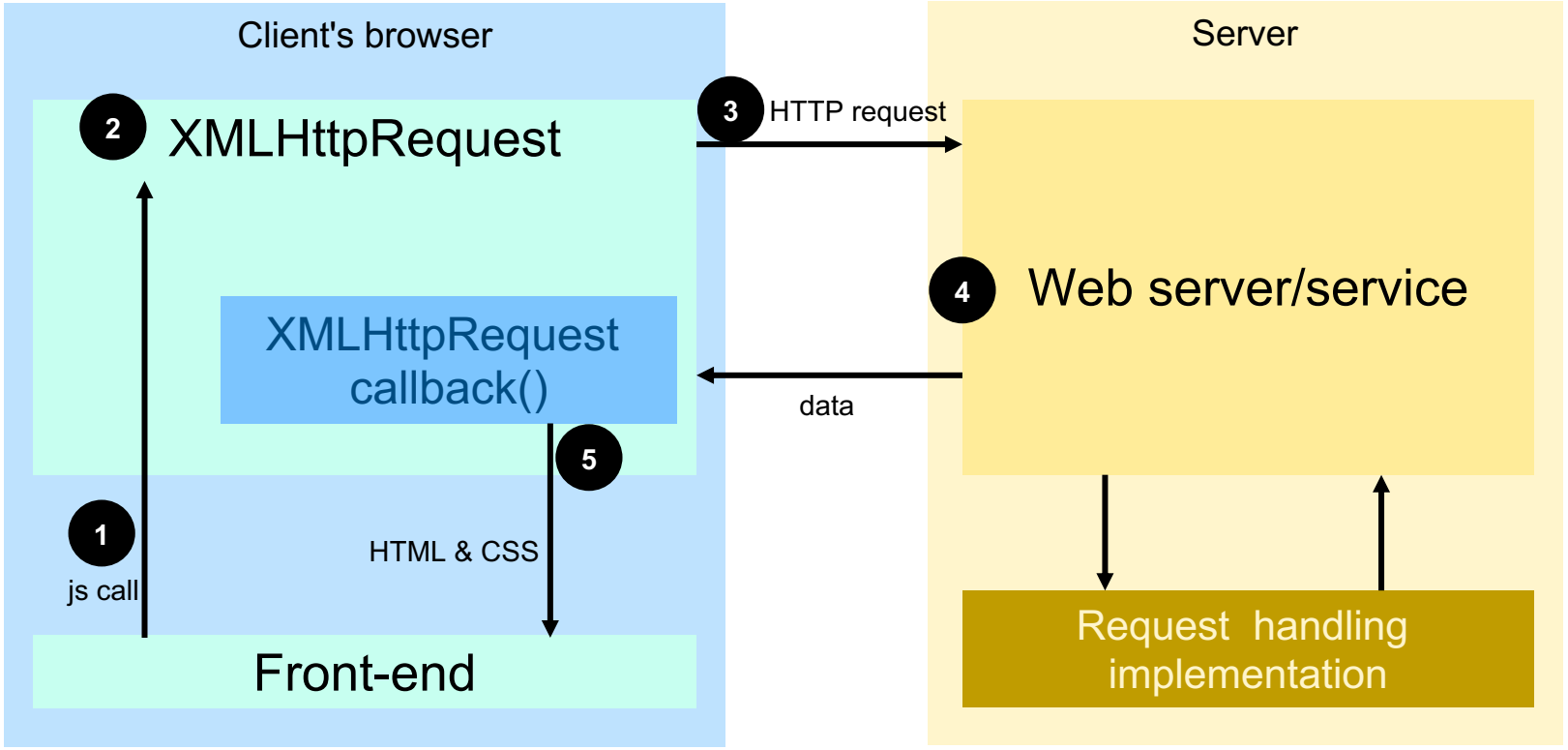
## XMLHttpRequest object can fetch files from a web server

- supported in IE5+, Safari, Firefox, Opera, Chrome
- it does this asynchronously, in the background
- the contents can be put into current web page using the DOM
- has various browser incompatibilities (prefer using jQuery!)

# Ajax requests

1) A frontend element requests to be updated.
2) An XMLHttpRequest object, is created, initialized with data and properties, and sent to the server.  The script / web page can continue after sending the request.
3) The server responds by sending a response.
4) When the response arrives from the server, a JavaScript function is triggered to act on the data returned.
5) This JavaScript response function typically refreshes the display using the DOM, with no need to reload or refresh the entire page.

# Ajax requests




## Scenarios

- XMLHttpRequest / DOM to accept response
- jQuery Ajax objects / jQuery to accept response

# Ajax request states

0: things are in the UNSENT state…the code understands that instance of XMLHttpRequest (xhr) has been created, but it's not doing anything.

1: things are in the OPEN state…the open() method has been invoked somewhere in our code and if you want to send any data to the server, you can do that with the send() method.

2: things are in the HEADERS_RECEIVED state…if you're downloading headers, the state will be set to 2 after they're finished downloading.

3: things are in the LOADING state…the data is loading.

4: things are in the DONE state…either the data has downloaded in full or there was an error during the download process.

# XMLHttpRequest / DOM

```
function getXHR() {
    let xhr;
    if (window.XMLHttpRequest) { // Browsers younger than 10 years...
        xhr = new XMLHttpRequest();
    } else {
        console.log("XMLHttpRequest() failed... Get some new browser, please...");
        return null;
    }
    return xhr;
};
```

# XMLHttpRequest / DOM

```
function ajax1Load() {
    function ajaxDataManager() {
        let text = document.getElementById("ajax1Area");
        if (ajaxDataToLoad.readyState === 4) {
            if (ajaxDataToLoad.status === 200) {
                text.innerHTML = ajaxDataToLoad.responseText;
            } else {
                console.log('Something went wrong with the request.'); } }
    };

    let ajaxDataToLoad = getXHR();
    if (ajaxDataToLoad) {
        ajaxDataToLoad.onreadystatechange = ajaxDataManager;            // what to do when ready
        ajaxDataToLoad.open("GET", "http://localhost:63342/dom-jquery-ajax-2/src/someLocalData.txt");
        ajaxDataToLoad.send(); }                                        // execution of call
    else {
        console.log("You MUST upgrade your browser NOW!!!"); }
}

 document.getElementById('ajax1Button').addEventListener("click", ajax1Load);
```

# jQuery / jQuery

```
$(document).ready(function() {

    $("#ajax3Button").click(function() {
        $.ajax({
            url: 'someLocalData.txt',
            type: 'GET',
            statusCode: {
                200: function () { console.log("GET completed successfully!"); }
            }
        }).done(function(response) {
            $("#ajax3Area").html(response);
        });

    });

});
```

# Resources

- https://www.digitalocean.com/community/tutorial_series/understanding-the-dom-document-object-model
- https://www.digitalocean.com/community/tutorials/an-introduction-to-jquery
- https://www.javatpoint.com/what-is-jquery
- https://www.javatpoint.com/ajax-example
- http://www.kaidez.com/ajax-tutorial/
- https://xhr.spec.whatwg.org/